

Frozen Executable Package ABI 設計仕様 v0.1

1. 最初に固定する設計判断

Frozen Packageには、二つの異なる表現を用意する。

バイナリ表現

ファイル保存、digest、別プロセス読込、長期互換性のためのABI

メモリ表現

Loaderによって検証・復号された、不変のC++オブジェクト

この二つを同一視してはならない。

次のようなC++型をそのままファイルへ書き出すことは禁止する。

```
std::vector<T>
std::string
std::variant<T...>
std::optional<T>
std::filesystem::path
ポインタ
size_t
bool
暗黙幅のenum
C++構造体全体のmemcpy
```

理由は、Compiler、標準ライブラリ、padding、alignment、32/64-bit環境によって表現が変わるためである。

バイナリは必ず、固定幅整数を使ってフィールド単位で書き込む。

```
writer.WriteU16(...);
writer.WriteU32(...);
writer.WriteU64(...);
writer.WriteBytes(...);
```

Readerも同様に、構造体へ直接reinterpret_castしない。

2. プロジェクト構造の微修正

設計憲法では `07_FrozenPackage` を一つのプロジェクトとしていたが、実際には二つに分ける方がよい。

```

07_FrozenPackageCore
    Package container
    Header
    Section table
    Digest
    Generic operation envelope
    Reader / Writer
    SourceにもD3D12にも依存しない

08_D3D12PackageSchema
    D3D12 Target Packageの固定スキーマ
    D3D12 headerには依存しない
    固定幅の独自enumとartifact型だけを持つ

09_PackageRuntime
10_D3D12Executor
11_PlatformWin32

```

依存は次になる。

```

06_D3D12TargetCompiler
    ↓
08_D3D12PackageSchema
    ↓
07_FrozenPackageCore

10_D3D12Executor
    ↓
08_D3D12PackageSchema
    ↓
07_FrozenPackageCore

```

TargetCompilerとExecutorは、同じPackage Schemaを共有する。

ただしExecutorはCompiler実装を参照しない。

`08_D3D12PackageSchema` は `d3d12.h` をincludeしない。D3D12 API型を直接保存する代わりに、Package用の安定した型を定義する。

3. 基本固定型

Package ABIで使うIDは、すべて32-bitの密な番号とする。

```

namespace sge::package
{
    inline constexpr std::uint32_t InvalidIndex = 0xffff'ffffu;
}

```

```

template<class Tag>
struct Id32
{
    std::uint32_t value = InvalidIndex;

    [[nodiscard]] constexpr bool IsValid() const noexcept
    {
        return value != InvalidIndex;
    }

    auto operator<=> (const Id32&) const = default;
};

struct ResourceTag;
struct AllocationTag;
struct ViewTag;
struct ShaderTag;
struct ProgramTag;
struct BindingLayoutTag;
struct ExecutableTag;
struct RasterCommandTag;
struct DynamicSlotTag;
struct ExternalSlotTag;
struct SurfaceSlotTag;
struct QueueTag;

using ResourceId = Id32<ResourceTag>;
using AllocationId = Id32<AllocationTag>;
using ViewId = Id32<ViewTag>;
using ShaderId = Id32<ShaderTag>;
using ProgramId = Id32<ProgramTag>;
using BindingLayoutId = Id32<BindingLayoutTag>;
using ExecutableId = Id32<ExecutableTag>;
using RasterCommandId = Id32<RasterCommandTag>;
using DynamicSlotId = Id32<DynamicSlotTag>;
using ExternalSlotId = Id32<ExternalSlotTag>;
using SurfaceSlotId = Id32<SurfaceSlotTag>;
using QueueId = Id32<QueueTag>;

struct IndexRange
{
    std::uint32_t first = 0;
    std::uint32_t count = 0;
};

struct Digest256
{
    std::array<std::byte, 32> bytes{};
    auto operator<=> (const Digest256&) const = default;
};

```

```
};  
}
```

IDには次の規則を課す。

各Table内で0から連続する
欠番を許さない
Table内でID順に配置する
InvalidIndexは参照なしを意味する
ファイル間で永続的な意味を持たない

Source側の `ResourceId` とPackage側の `ResourceId` は別型にする。

CompilerがSource IDからPackage IDへの対応表を持つ。

4. Blob参照

大きなデータはTable record内へ直接埋め込まず、Blob sectionへ置く。

```
enum class SectionKind : std::uint32_t;  
  
struct BlobRef  
{  
    SectionKind section = SectionKind{};  
    std::uint32_t reserved = 0;  
    std::uint64_t offset = 0;  
    std::uint64_t size = 0;  
};
```

`offset` は指定Sectionの先頭からの相対offsetとする。

ファイル全体からのoffsetにしない理由は、Section単位の再配置と検証を容易にするためである。

BlobRefは必ず次を満たす。

reserved == 0
offset + sizeでoverflowしない
参照先Sectionの範囲内
要求alignmentを満たす
実行SectionからDebug sectionを参照しない

5. Packageファイル全体

ファイル拡張子は仮に次とする。

```
.sgep
Semantic GPU Executable Package
```

任意のSource対応情報は別ファイルにする。

```
.sgedbg
Semantic GPU Debug Map
```

`.sgep` はSourceなしで実行可能でなければならない。

`.sgedbg` は `.sgep` のexecution digestを参照し、Source Work名、元ファイル、診断位置などを保持する。

6. Package Header

Headerは176 bytes固定とする。

すべてlittle endianで保存する。

Offset	Size	Field
0x00	8	magic = "SGE2PKG\0"
0x08	2	formatMajor
0x0A	2	formatMinor
0x0C	2	headerBytes
0x0E	2	sectionDescriptorBytes
0x10	4	targetKind
0x14	4	targetSchemaVersion
0x18	4	minimumRuntimeVersion
0x1C	4	flags
0x20	4	endianTag = 0x01020304
0x24	4	digestAlgorithm
0x28	8	fileBytes
0x30	8	sectionTableOffset
0x38	4	sectionCount
0x3C	4	reserved0

0x40	32	targetProfileDigest
0x60	32	executionDigest
0x80	32	fileDigest
0xA0	16	reserved1

Headerの総サイズは `0xB0 = 176 bytes` である。

初期値は次とする。

```
formatMajor = 1
formatMinor = 0
headerBytes = 176
sectionDescriptorBytes = 80
targetKind = 1           // D3D12
targetSchemaVersion = 1
digestAlgorithm = 1      // SHA-256
sectionTableOffset = 176
```

reserved領域はすべて0でなければならない。

三種類のdigest

targetProfileDigest

D3D12 Target Profile sectionだけをcanonical encodingしたdigest。

DeviceがPackage要求を満たすか、cache identityを分けるために使う。

executionDigest

実行結果へ影響する全Sectionから計算する。

次を含む。

```
Resource
Initial data
Shader binaries
Binding layout
Executable
Descriptor plan
Invocation schema
Operation stream
Target profile
```

次は含めない。

DebugMap
Source name
Compilerログ
Timestamp
Provenance

fileDigest

ファイル全体から計算する。

計算時だけHeader内の `fileDigest` 32 bytesを0として扱う。

ファイル破損検出に使う。

7. Section Descriptor

各Section descriptorは80 bytes固定とする。

Offset	Size	Field
0x00	4	sectionKind
0x04	2	schemaVersion
0x06	2	descriptorBytes
0x08	4	flags
0x0C	4	alignment
0x10	8	fileOffset
0x18	8	storedBytes
0x20	8	logicalBytes
0x28	4	elementCount
0x2C	4	elementStride
0x30	32	sectionDigest

Section tableは `sectionKind` の昇順に並べる。

同じkindを二つ持つことは、V1では禁止する。

Section flagsは少なくとも次を持つ。

```
enum class SectionFlags : std::uint32_t
{
    None = 0,
```

```

Required      = 1u << 0,
ExecutionAffecting = 1u << 1,
DebugOnly     = 1u << 2,
OpaqueToCore  = 1u << 3
};

```

V1ではSection圧縮を禁止する。

圧縮方式の違いによってPackage bytesの決定性が崩れるためである。

配布時の圧縮は `.sgep` の外側でZIP等を使う。

8. Section Kind

Core containerが理解するSectionと、D3D12 schemaが理解するSectionを番号帯で分ける。

```

enum class SectionKind : std::uint32_t
{
    Manifest          = 0x0000'0001,
    InvocationSchema   = 0x0000'0002,
    OperationStreamTable = 0x0000'0003,
    OperationTable     = 0x0000'0004,
    OperationPayload    = 0x0000'0005,
    InitialData        = 0x0000'0006,
    ShaderData         = 0x0000'0007,
    NativeObjectData   = 0x0000'0008,

    D3D12TargetProfile = 0x0000'1000,
    D3D12ResourceTable = 0x0000'1001,
    D3D12AllocationTable = 0x0000'1002,
    D3D12ViewTable     = 0x0000'1003,
    D3D12ShaderTable    = 0x0000'1004,
    D3D12ProgramTable   = 0x0000'1005,
    D3D12BindingLayoutTable = 0x0000'1006,
    D3D12ExecutableTable = 0x0000'1007,
    D3D12RasterCommandTable = 0x0000'1008,
    D3D12DescriptorPlan  = 0x0000'1009,
    D3D12DynamicSlotTable = 0x0000'100A,
    D3D12ExternalSlotTable = 0x0000'100B,
    D3D12SurfaceSlotTable = 0x0000'100C,

    StringTable        = 0x0000'7F00,
    DebugMap           = 0x0000'7F01,
    Provenance         = 0x0000'7F02
};

```

Core ReaderはD3D12 Sectionの内部意味を理解しなくてよい。

ただしoffset、size、overlap、digestは検証する。

D3D12 schema validatorが、D3D12 Section内の参照整合性を検証する。

9. Manifest

ManifestはPackage全体の入口である。

```
struct PackageManifest
{
    std::uint32_t resourceCount = 0;
    std::uint32_t allocationCount = 0;
    std::uint32_t viewCount = 0;
    std::uint32_t shaderCount = 0;
    std::uint32_t programCount = 0;
    std::uint32_t executableCount = 0;
    std::uint32_t rasterCommandCount = 0;

    std::uint32_t dynamicSlotCount = 0;
    std::uint32_t externalSlotCount = 0;
    std::uint32_t surfaceSlotCount = 0;

    std::uint32_t loadOperationStream = InvalidIndex;
    std::uint32_t frameOperationStream = InvalidIndex;

    std::uint32_t flags = 0;
};
```

これはメモリ上のdecoded型であり、ファイルではフィールドごとに書き込む。

10. D3D12 Target Profile

Target Profileは「Compilerが何を前提としてPackageを生成したか」を記録する。

```
namespace sge::package::d3d12_v1
{
    enum class BarrierModel : std::uint32_t
    {
        Legacy = 1,
        Enhanced = 2
    };

    enum class ShaderBinaryFormat : std::uint16_t
    {
        Dxbc = 1,
```

```

    Dxil = 2
};

struct TargetProfile
{
    std::uint32_t minimumFeatureLevel = 0;
    std::uint16_t shaderModelMajor = 0;
    std::uint16_t shaderModelMinor = 0;
    std::uint16_t rootSignatureMajor = 0;
    std::uint16_t rootSignatureMinor = 0;

    BarrierModel barrierModel = BarrierModel::Legacy;

    std::uint32_t framesInFlight = 1;
    std::uint32_t directQueueCount = 1;
    std::uint32_t computeQueueCount = 0;
    std::uint32_t copyQueueCount = 0;

    std::uint32_t rtvDescriptorCount = 0;
    std::uint32_t dsvDescriptorCount = 0;
    std::uint32_t shaderDescriptorCount = 0;
    std::uint32_t samplerDescriptorCount = 0;

    std::uint64_t requiredFeatureBits0 = 0;
    std::uint64_t requiredFeatureBits1 = 0;
};
}

```

初期Triangle sliceでは次になる。

```

framesInFlight = 1
directQueueCount = 1
computeQueueCount = 0
copyQueueCount = 0
rtvDescriptorCount = Surface image数
dsvDescriptorCount = 0
shaderDescriptorCount = 0
samplerDescriptorCount = 0
barrierModel = Legacy

```

初期Packageでは特定AdapterのLUIDやDriver versionへ固定しない。

将来、Device固有最適化Packageを導入する場合は、別Target profile versionとして追加する。

11. Resource Artifact

Package ResourceはSource Resource declarationのコピーではない。

```

enum class ResourceKind : std::uint16_t
{
    Buffer = 1,
    Texture1D = 2,
    Texture2D = 3,
    Texture3D = 4,
    SurfaceImage = 5
};

enum class ResourceOrigin : std::uint16_t
{
    PackageOwned = 1,
    External = 2,
    Surface = 3
};

enum class RebuildPolicy : std::uint16_t
{
    RecreateFromPackage = 1,
    RequireExternalRebind = 2,
    RuntimeManaged = 3
};

enum class ExtentMode : std::uint16_t
{
    Fixed = 1,
    SurfaceRelative = 2
};

```

Resource record V1は96 bytesとする。

Offset	Size	Field
0x00	4	resourceId
0x04	2	resourceKind
0x06	2	origin
0x08	2	rebuildPolicy
0x0A	2	extentMode
0x0C	4	flags
0x10	4	physicalInstanceCount
0x14	4	allocationId
0x18	4	format
0x1C	4	usageFlags
0x20	8	initialState
0x28	8	sizeBytes

0x30	4	width
0x34	4	height
0x38	2	depthOrArraySize
0x3A	2	mipLevels
0x3C	2	sampleCount
0x3E	2	planeCount
0x40	8	initialDataOffset
0x48	8	initialDataSize
0x50	4	firstView
0x54	4	viewCount
0x58	8	reserved

Bufferでは、

```
sizeBytes > 0
width = height = 0
mipLevels = 0
```

Textureでは、

```
sizeBytes = 0
width > 0
height > 0
mipLevels > 0
```

SurfaceImageでは、

```
origin = Surface
rebuildPolicy = RuntimeManaged
extentMode = SurfaceRelative
physicalInstanceCount = 0
allocationId = InvalidIndex
initialDataSize = 0
```

12. Resource State

D3D12のnative numeric stateを、そのままPackageへ保存してはならない。

特にD3D12では **COMMON** と **PRESENT** が同じ数値であり、単なるbit値では意味を区別できない。

Packageでは次のように表す。

```
enum class StateClass : std::uint16_t
{
    Common = 1,
    Present = 2,
    Explicit = 3
};

struct ResourceState
{
    StateClass stateClass = StateClass::Common;
    std::uint16_t reserved = 0;
    std::uint32_t explicitBits = 0;
};
```

`Explicit` のbitsはPackage schemaで定義した固定bitを使う。

```
enum class ExplicitStateBits : std::uint32_t
{
    None = 0,
    VertexBuffer = 1u << 0,
    ConstantBuffer = 1u << 1,
    IndexBuffer = 1u << 2,
    RenderTarget = 1u << 3,
    DepthWrite = 1u << 4,
    DepthRead = 1u << 5,
    ShaderRead = 1u << 6,
    UnorderedWrite = 1u << 7,
    CopySource = 1u << 8,
    CopyDestination = 1u << 9,
    IndirectArgument = 1u << 10
};
```

ExecutorはこのPackage stateをD3D12 native stateへ機械的に変換する。

Compilerがstateを決め、Executorは選択しない。

13. View Artifact

ViewはResourceの利用方法を固定する。

```
enum class ViewClass : std::uint16_t
{
    VertexBuffer = 1,
    IndexBuffer = 2,
```

```

ConstantBuffer = 3,
ShaderResource = 4,
UnorderedAccess = 5,
RenderTarget = 6,
DepthStencil = 7,
CopySource = 8,
CopyDestination = 9,
PresentSource = 10
};

```

第一世代のように、BackendがBinding種別からDescriptor種別を推測してはならない。

Compilerが必要なViewをすべて作成する。

概念的なdecoded型は次になる。

```

struct ResourceViewArtifact
{
    ViewId id;
    ResourceId resource;
    ViewClass viewClass;

    std::uint32_t format = 0;
    std::uint32_t flags = 0;

    std::uint64_t byteOffset = 0;
    std::uint64_t byteSize = 0;
    std::uint32_t strideBytes = 0;

    std::uint16_t firstMip = 0;
    std::uint16_t mipCount = 0;
    std::uint16_t firstArrayLayer = 0;
    std::uint16_t arrayLayerCount = 0;
    std::uint16_t firstPlane = 0;
    std::uint16_t planeCount = 0;

    std::uint32_t descriptorHeapClass = 0;
    std::uint32_t descriptorIndex = InvalidIndex;
    std::uint32_t descriptorInstanceStride = 0;
};

```

Descriptorが必要なViewは、Descriptor indexまでCompilerが固定する。

Executorがheap内の空き場所を探してはならない。

14. Allocation Artifact

CompilerがResource配置を決める。

```
enum class AllocationKind : std::uint16_t
{
    Committed = 1,
    Placed = 2,
    RuntimeManaged = 3
};

enum class HeapClass : std::uint16_t
{
    DefaultBuffer = 1,
    DefaultTexture = 2,
    RenderTargetOrDepth = 3,
    Upload = 4,
    Readback = 5
};

struct AllocationArtifact
{
    AllocationId id;
    AllocationKind kind;
    HeapClass heapClass;

    std::uint32_t flags = 0;
    std::uint32_t physicalInstanceCount = 1;

    std::uint64_t sizeBytes = 0;
    std::uint64_t alignment = 0;

    std::uint32_t aliasGroup = InvalidIndex;
};
```

初期TriangleではVertex BufferをCommitted resourceとしてよい。

Aliasingは後続Sliceで追加する。

15. Shader Artifact

Shader compileはTargetCompilerで完了させる。

```
enum class ShaderStage : std::uint16_t
{
    Vertex = 1,
```

```

    Pixel = 2,
    Compute = 3
};

struct ShaderArtifact
{
    ShaderId id;
    ShaderStage stage;
    ShaderBinaryFormat format;

    std::uint16_t shaderModelMajor = 0;
    std::uint16_t shaderModelMinor = 0;

    std::uint32_t flags = 0;

    BlobRef bytecode;
    Digest256 bytecodeDigest;
};

```

Packageには次を入れない。

```

HLSL path
HLSL source
entry point名
include path
compile command

```

Compiler toolchain情報は、任意のProvenance sectionへ入れる。

実行digestにはShader binaryそのものが入る。

初期Shader形式

ABIはDXBCとDXILの両方を表現可能にする。

ただし一つのTarget profile内で使用する形式を固定する。

最初のTriangle sliceでは、設計検証をDXC配備問題から分離するため、DXBC Shader Model 5.1を使用してもよい。

DXILへ移行する場合は、同じPackageを黙って別binaryへ変えず、Target profileを変更する。

16. Binding Layout

BackendでRoot Signature方針を決めない。

TargetCompilerがRoot Signatureをserializeし、そのbinaryをPackageへ保存する。

```
struct BindingLayoutArtifact
{
    BindingLayoutId id;

    std::uint16_t rootSignatureMajor = 0;
    std::uint16_t rootSignatureMinor = 0;
    std::uint32_t flags = 0;

    IndexRange parameterRange;
    IndexRange descriptorRange;
    IndexRange staticSamplerRange;

    BlobRef serializedRootSignature;
    Digest256 layoutDigest;
};
```

Packageには、

```
検証用のcanonical binding layout
実行用のserialized root signature binary
```

の両方を持たせる。

Executorは `D3D12SerializeRootSignature` を呼ばない。

`CreateRootSignature` に保存済みbinaryを渡すだけとする。

17. Program Artifact

```
enum class ProgramKind : std::uint16_t
{
    Raster = 1,
    Compute = 2
};

struct ProgramArtifact
{
    ProgramId id;
    ProgramKind kind;
    std::uint16_t flags = 0;

    ShaderId vertexShader;
    ShaderId pixelShader;
    ShaderId computeShader;
```

```
BindingLayoutId bindingLayout;  
Digest256 interfaceDigest;  
};
```

Raster Programでは、

```
vertexShader valid  
pixelShader valid  
computeShader invalid
```

Compute Programでは逆になる。

18. Executable Artifact

ProgramとPipeline specializationを分ける。

同じProgramでも、Attachment formatやRaster stateが異なれば別Executableになる。

```
struct RasterExecutableArtifact  
{  
    ExecutableId id;  
    ProgramId program;  
    BindingLayoutId bindingLayout;  
  
    IndexRange vertexElementRange;  
    IndexRange colorFormatRange;  
  
    std::uint32_t depthFormat = 0;  
  
    std::uint32_t primitiveTopology = 0;  
    std::uint32_t primitiveTopologyType = 0;  
  
    std::uint32_t rasterStateId = InvalidIndex;  
    std::uint32_t blendStateId = InvalidIndex;  
    std::uint32_t depthStateId = InvalidIndex;  
  
    std::uint32_t sampleCount = 1;  
    std::uint32_t sampleQuality = 0;  
  
    Digest256 specializationDigest;  
};
```

Executorはこの記述からPSOを作成する。

PSOの選択肢を比較したり、stateを補完したりしない。

19. Raster Command

Operationに大量の描画情報を直接埋めず、Command artifactを参照する。

```
struct RasterCommandArtifact
{
    RasterCommandId id;
    ExecutableId executable;

    IndexRange vertexViewRange;
    ViewId indexView;

    IndexRange colorAttachmentRange;
    ViewId depthAttachment;

    std::uint32_t vertexCount = 0;
    std::uint32_t instanceCount = 1;
    std::uint32_t firstVertex = 0;
    std::uint32_t firstInstance = 0;

    std::uint32_t indexCount = 0;
    std::uint32_t firstIndex = 0;
    std::int32_t baseVertex = 0;

    std::uint32_t viewportId = InvalidIndex;
    std::uint32_t scissorId = InvalidIndex;
    std::uint32_t attachmentOperationId = InvalidIndex;
};
```

初期Triangleでは、

```
vertexViewRange.count = 1
indexView = InvalidIndex
colorAttachmentRange.count = 1
depthAttachment = InvalidIndex
vertexCount = 3
instanceCount = 1
```

となる。

20. Invocation Schema

Runtime入力は内部ResourceIdではなく、公開Slot IDを使う。

Dynamic data slot

```
struct DynamicDataSlotArtifact
{
    DynamicSlotId id;
    ResourceId destinationResource;

    std::uint64_t destinationOffset = 0;
    std::uint64_t requiredBytes = 0;
    std::uint32_t requiredAlignment = 1;
    std::uint32_t flags = 0;
};
```

External Resource slot

```
struct ExternalResourceSlotArtifact
{
    ExternalSlotId id;
    ResourceId resource;

    std::uint32_t requiredKind = 0;
    std::uint32_t requiredFormat = 0;

    std::uint64_t minimumBytes = 0;

    ResourceState requiredIncomingState;
    ResourceState guaranteedOutgoingState;

    std::uint32_t synchronizationContract = 0;
    std::uint32_t flags = 0;
};
```

重要な変更として、FrameInvocationからincoming/outgoing stateの自由なoverrideを除く。

状態契約はPackageが固定する。

```
Package:
    required incoming state
    guaranteed outgoing state

Caller:
    この契約を満たすResourceとcompletionを渡す
```

可変stateを将来許す場合は、専用のState parameter契約として追加する。

Undefined による暗黙推測は行わない。

Surface slot

```
struct SurfaceSlotArtifact
{
    SurfaceSlotId id;
    ResourceId imageResource;

    std::uint32_t requiredFormat = 0;
    ResourceState acquiredState;
    ResourceState presentedState;

    std::uint32_t flags = 0;
};
```

初期Triangleでは、

```
acquiredState = Present
presentedState = Present
```

であり、Frame operation内で、

```
Present → RenderTarget
RenderTarget → Present
```

を明示する。

21. Operation Stream

Operationは二つのstreamへ分ける。

```
Load stream
    Package load時
    Device recovery時
    Resizeによる再物質化時

Frame stream
    毎フレーム実行
```

Backend独自のPreparation順序は持たない。

Stream record

```
enum class OperationStreamKind : std::uint32_t
{
    Load = 1,
    Frame = 2
};

struct OperationStreamArtifact
{
    OperationStreamKind kind;
    std::uint32_t firstOperation = 0;
    std::uint32_t operationCount = 0;
    std::uint32_t flags = 0;
};
```

16 bytes固定とする。

Operation record

Operation recordは24 bytes固定とする。

Offset	Size	Field
0x00	4	opcode
0x04	2	operationVersion
0x06	2	flags
0x08	4	queueId
0x0C	4	payloadBytes
0x10	8	payloadOffset

`payloadOffset` はOperationPayload sectionからの相対offset。

Queueと無関係なOperationでは `queueId = InvalidIndex` とする。

Opcodeは上位16-bitをdomain、下位16-bitをcodeとする。

```
0x0001xxxx = Package Core
0x0002xxxx = D3D12 target
```

未知のOperationは必ず拒否する。

実行へ影響するOperationを無視して続行してはならない。

22. D3D12 Operation V1

初期候補を次とする。

```
enum class D3D12OperationCode : std::uint32_t
{
    CreateDescriptorHeaps = 0x0002'0001,
    CreateResource      = 0x0002'0002,
    UploadBuffer        = 0x0002'0003,
    CreateRootSignature = 0x0002'0004,
    CreateGraphicsPipeline = 0x0002'0005,
    InitializeState     = 0x0002'0006,

    AcquireSurfacelImage = 0x0002'0100,
    AcquireExternal      = 0x0002'0101,
    WaitExternal         = 0x0002'0102,

    BeginQueueBatch     = 0x0002'0200,
    Transition          = 0x0002'0201,
    ActivateAlias       = 0x0002'0202,
    ExecuteRaster       = 0x0002'0203,
    ExecuteCompute      = 0x0002'0204,
    ExecuteCopy         = 0x0002'0205,
    EndQueueBatch       = 0x0002'0206,
    SignalQueue         = 0x0002'0207,

    ReleaseExternal     = 0x0002'0300,
    PresentSurface      = 0x0002'0301
};
```

初期Triangleに不要なOperationはSchemaには予約しても、Compilerは出力しない。

23. Transition Operation

Transitionはbeforeとafterを両方持つ。

```
struct TransitionPayload
{
    ViewId view;
    std::uint32_t flags = 0;

    ResourceState before;
    ResourceState after;
};
```

ExecutorがResourceの現在状態を推測してはならない。

Runtimeは内部追跡状態と `before` が一致することを確認する。

一致しない場合は、Packageまたは実行状態の破損として停止する。

24. Load Stream

初期TriangleのLoad streamは概念的に次になる。

```
CreateDescriptorHeaps  
  
CreateResource(VertexBuffer)  
UploadBuffer(VertexBuffer)  
InitializeState(VertexBuffer, VertexBufferState)  
  
CreateRootSignature(MainProgram)  
CreateGraphicsPipeline(MainExecutable)
```

Surface imageはSwapChainが所有するため、Load streamで作成しない。

Device loss後は同じLoad streamを再実行する。

これによりPackage-owned objectの再物質化が同一手順になる。

25. Frame Stream

初期TriangleのFrame streamは次になる。

```
AcquireSurfaceImage(MainSurface)  
  
BeginQueueBatch(DirectQueue)  
  
Transition(  
    PresentationView,  
    Present,  
    RenderTarget)  
  
ExecuteRaster(TriangleCommand)  
  
Transition(  
    PresentationView,  
    RenderTarget,  
    Present)  
  
EndQueueBatch(DirectQueue)
```


SignalQueue(DirectQueue)

PresentSurface(MainSurface)

PresentはBackendがOperation列の外で勝手に行わない。

Packageに `PresentSurface` がなければPresentしない。

26. FrameInvocation

正式なRuntime入力型は次の方向とする。

```
struct DynamicDataBinding
{
    DynamicSlotId slot;
    std::span<const std::byte> bytes;
};

struct ExternalResourceBinding
{
    ExternalSlotId slot;
    std::shared_ptr<IExternalResource> resource;
    std::shared_ptr<ICompletionToken> availableAfter;
};

struct FrameInvocation
{
    std::uint64_t frameNumber = 0;

    std::span<const DynamicDataBinding> dynamicData;
    std::span<const ExternalResourceBinding> externalResources;
};
```

Resource stateのoverrideは持たない。

RuntimeはSubmit前に次を検証する。

必須slotがすべて存在する
同じslotが重複していない
未知slotがない
Dynamic dataのサイズが完全一致する
External Resourceの種類とshapeが一致する
Resourceが現在のdevice epochに属する

27. FrameSubmission

```
struct QueueCompletion
{
    QueueId queue;
    std::uint64_t value = 0;
    std::shared_ptr<ICompletionToken> completion;
};

struct ExternalRelease
{
    ExternalSlotId slot;
    std::shared_ptr<ICompletionToken> safeAfter;
};

struct FrameSubmission
{
    std::uint64_t deviceEpoch = 0;
    std::vector<QueueCompletion> queues;
    std::vector<ExternalRelease> releasedExternalResources;
};
```

外部Resourceの利用者は、 `ExternalRelease.safeAfter` を使ってGPU完了を待てる。

28. メモリ上のFrozenExecutablePackage

Loaderが返すC++オブジェクトは不変にする。

```
class FrozenExecutablePackage final
{
public:
    [[nodiscard]] const PackageHeader& Header() const noexcept;
    [[nodiscard]] std::span<const SectionView> Sections() const noexcept;

    [[nodiscard]] const Digest256& ExecutionDigest() const noexcept;
    [[nodiscard]] TargetKind Target() const noexcept;

private:
    friend class PackageReader;

    std::shared_ptr<const PackageStorage> storage_;
    PackageHeader header_;
    std::vector<SectionView> sections_;
};
```

`PackageStorage` は元ファイルbytesを所有する。

Target schema decoderはこれを受け取る。

```
class D3D12PackageView final
{
public:
    static base::Result<D3D12PackageView, PackageError> Decode(
        const FrozenExecutablePackage& package);

    [[nodiscard]] const TargetProfile& Profile() const noexcept;
    [[nodiscard]] std::span<const ResourceArtifact> Resources() const noexcept;
    [[nodiscard]] std::span<const ResourceViewArtifact> Views() const noexcept;
    [[nodiscard]] std::span<const ShaderArtifact> Shaders() const noexcept;
    [[nodiscard]] std::span<const ProgramArtifact> Programs() const noexcept;
    [[nodiscard]] std::span<const RasterExecutableArtifact>
        Executables() const noexcept;
    [[nodiscard]] std::span<const OperationArtifact>
        LoadOperations() const noexcept;
    [[nodiscard]] std::span<const OperationArtifact>
        FrameOperations() const noexcept;
};
```

DecoderはSource IRやCompiler reportを返さない。

29. Readerの検証順序

Package Readerは次の順序で検証する。

1. 最小ファイルサイズ
2. Magic
3. Endian tag
4. Container major/minor version
5. Header size
6. File size一致
7. Section table範囲
8. Section descriptor size
9. Section kind順序
10. 重複Section
11. Alignment
12. offset + size overflow
13. Section間overlap
14. 必須Section存在
15. Section digest
16. File digest
17. Execution digest
18. element stride
19. Table ID連続性
20. 全参照の範囲

- 21. BlobRef範囲
- 22. enum値
- 23. Target profile互換性
- 24. Operation stream構造
- 25. Resource state整合性
- 26. Invocation schema整合性

検証途中でD3D12 objectを作成しない。

Package全体の静的検証に成功してから物質化を開始する。

30. Canonical serialization

同一入力からbyte単位で同じPackageを生成するため、Writerに次の規則を課す。

SectionはSectionKind昇順
Table recordはID昇順
IDは0から連続
MapやSetはkey昇順へ変換
文字列はUTF-8
Debug文字列は実行Sectionへ入れない
Blobは所有者ID順
Texture dataはmip、array、plane順
padding byteはすべて0
reserved fieldはすべて0
Timestampを入れない
Random UUIDを入れない
Pointer値を入れない
Compiler process固有値を入れない

FloatはIEEE 754 binary32のbit patternとして保存する。

次も規定する。

NaNを実行用数値として拒否する
-0.0は+0.0へ正規化する
無限大を許すfieldと禁止するfieldを明示する

Compiler versionやGit commitを残したい場合はProvenance sectionに入れる。

Execution digestには含めない。

同一Compile optionならProvenanceを含めたfile bytesも決定的にする。

31. Versioning

四種類のversionを分ける。

Container format version
Header、Section tableの形式

Target schema version
D3D12 artifact全体の契約

Section schema version
個別Table recordの契約

Operation version
個別Opcode payloadの契約

Major version

既存Readerが安全に解釈できない変更。

fieldの意味変更
既存Operationの意味変更
digest計算規則変更
ID規則変更

Minor version

既存Readerが無視できる追加。

Optional section追加
record末尾へのfield追加
新しいDebug情報

未知Sectionは、

Requiredなら拒否
Optionalなら無視

未知Operationは常に拒否する。

32. Package Error

```
enum class PackageErrorCode
{
    BadMagic,
    UnsupportedContainerVersion,
    UnsupportedTarget,
    UnsupportedTargetSchema,
    WrongEndianness,
    FileSizeMismatch,
    InvalidSectionTable,
    DuplicateSection,
    MissingRequiredSection,
    UnknownRequiredSection,
    SectionOutOfBounds,
    SectionOverlap,
    InvalidAlignment,
    DigestMismatch,
    InvalidRecordStride,
    InvalidIdSequence,
    InvalidReference,
    InvalidBlobReference,
    InvalidEnumValue,
    InvalidTargetProfile,
    InvalidOperationStream,
    InvalidInvocationSchema,
    TargetCapabilityMismatch
};

struct PackageError
{
    PackageErrorCode code;
    SectionKind section{};
    std::uint32_t recordIndex = InvalidIndex;
    std::uint32_t operationIndex = InvalidIndex;
    std::string message;
};
```

Runtime診断はPackageのSectionとrecord indexを基準にする。

Source位置は任意のDebug sidecarで対応付ける。

33. 最初のTriangle Packageの実体

最初の `.sgcp` は概念的に次を含む。

Header

Section Table

Manifest

D3D12 Target Profile

Resource Table

0: TriangleVertexBuffer

1: MainSurfaceImage

Allocation Table

0: TriangleVertexAllocation

View Table

0: TriangleVertexView

1: MainSurfaceRenderTargetView

2: MainSurfacePresentView

Shader Table

0: TriangleVertexShader

1: TrianglePixelShader

Program Table

0: TriangleProgram

Binding Layout Table

0: EmptyRootSignature

Executable Table

0: TriangleGraphicsExecutable

Raster Command Table

0: DrawTriangle

Surface Slot Table

0: MainSurface

Load Operation Stream

Frame Operation Stream

InitialData

Triangle vertex bytes

ShaderData

Vertex shader binary

Pixel shader binary

NativeObjectData

Serialized root signature

Packageに存在しないもの：

```
Triangle.hlsl  
shaderPath  
VSMMainというentry文字列  
PSMainというentry文字列  
SemanticGraph  
Compiler analysis  
ExecutionPlan  
D3D12Backend用の推測値
```

34. 最初の完成試験

Triangle sliceでは、次の試験を必須とする。

Byte determinism

同一SourceとTarget Profileから二回Compileする。

```
first.sgep  
second.sgep
```

をbyte単位で比較し、完全一致させる。

Round trip

```
Compile  
Serialize  
Read  
Validate  
Serialize again
```

でbytesが一致することを確認する。

Corruption

次を一つずつ破壊し、必ずReaderが拒否することを確認する。

```
Magic  
File size  
Section offset  
Section size  
Section digest
```


Shader byte
Resource ID
View reference
Operation payload offset
Operation code
Execution digest

Source independence

1. TriangleをPackageへCompile
2. Packageを保存
3. HLSLを削除または別フォルダへ移動
4. Compiler DLLを含まないExecutorアプリを起動
5. Packageだけを読み込む
6. Triangleを表示

Project dependency

D3D12ExecutorのProjectReferenceに次が存在しないことを確認する。

SemanticModel
SemanticBuilder
SemanticAnalysis
D3D12TargetCompiler
ShaderCompiler
Frontend

35. 実装開始順序

最初にD3D12描画を作らない。

次の順序で作る。

段階1

00_Base
07_FrozenPackageCore
32_PackageContractTests

段階2

Header Writer / Reader
Section Writer / Reader
SHA-256
Canonical alignment
File digest

Corruption tests

段階3

08_D3D12PackageSchema
Target Profile
Resource/View/Shader/Program/Executable record
Schema validator

段階4

手書きのTriangle Packageを生成
まだSemantic Compilerは作らない
Package round-tripを完成

段階5

10_D3D12Executor
手書きPackageを実行
SourceなしでTriangle表示

段階6

SemanticModel
SemanticBuilder
SemanticAnalysis

段階7

D3D12TargetCompiler
Semantic Triangleから同じPackageを生成

段階8

手書きPackage生成器を削除

この順序により、

Package契約が上流実装の都合で変形する

ことを防ぐ。

まず最終国境を独立して成立させ、その後Source側を接続する。

36. この仕様で確定したこと

第二世代のFrozen Packageは、単なるC++オブジェクトではない。

version付き
Section化
固定幅
決定的

digest付き
Source非依存
Compiler非依存
Target明示
単独検証可能
別プロセス読込可能
Device再作成可能

な実行成果物である。

Compilerはこの形式を生成する。

Runtimeはこの形式を検証・保持する。

D3D12Executorはこの形式をD3D12へ機械的に写像する。

Backendが情報不足を補う設計は、Frozen Packageの不備として扱う。